

Problema 3. Combinar cifras y operaciones

Nuestra interpretación del problema es la siguiente. Se dispone de nueve cifras: $\{1,2,3,4,5,6,7,8,9\}$ y de los cuatro operadores aritméticos básicos $\{+, -, \times, /\}$.

Se pide formar expresiones alternando cifras y operadores, combinándolos alternativamente sin repetir ninguno de ellos para obtener un determinado valor. De acuerdo con el ejemplo proporcionado (para obtener el 4: $4+2-6/3 \times 1 = 4$), entendemos que:

- Se utilizan solo 5 cifras distintas de entre las 9 disponibles.
- Las cifras utilizadas son distintas, no se repiten cifras.
- Se utilizan los 4 operadores básicos,
- Cada uno de los operadores se utiliza solo una vez.
- Las cifras y los operadores aparecen alternados.
- La precedencia de las operaciones es la habitual y no se utilizan paréntesis

Una expresión válida tendrá la estructura:

$$d_1 * op_1 * d_2 * op_2 * d_3 * op_3 * d_4 * op_4 * d_5 * op_5$$

siendo:

- d_i : son cifras distintas elegidas de $\{1, \dots, 9\}$.
- op_i : son los operadores $+, -, \times, /$, utilizados exactamente una vez cada uno.

Las restricciones que debe cumplir cualquier expresión son:

1. No repetir cifras: Cada cifra puede utilizarse como máximo una vez.
2. No repetir operadores: Cada operador debe utilizarse exactamente una vez.
3. Alternancia: Las cifras y operadores deben aparecer alternados.
4. Precedencia de operaciones: precedencia habitual de las reglas aritméticas: primero multiplicación y división y luego suma y resta.

- ¿Qué valor máximo y mínimo se pueden obtener según las condiciones del problema?

Como el número total de expresiones posibles: $VR(9,5) \cdot 4! = 15\,120 \cdot 24 = 362880$ (ver respuesta a la pregunta 2), no es excesivamente alto hemos evaluado todas las soluciones posibles que cumplen las restricciones (cinco cifras distintas tomadas de 1 a 9 y los cuatro operadores utilizados exactamente una vez), hemos obtenido que:

el menor valor posible (mínimo) es: $1 + \frac{4}{2} - 8 \cdot 9 = -69$

y el mayor valor posible (máximo) es: $\frac{7}{1} - 2 + 8 \cdot 9 = 77$.

- ¿Es posible encontrar todos los valores enteros posibles entre dicho mínimo y máximo?

Sí, es posible obtener todos los valores enteros entre -69 y 77.

Al almacenar todos los resultados enteros generados por las 362880 expresiones válidas, se obtienen exactamente 147 valores enteros distintos.

Por otro lado tenemos que el intervalo $[-69,77]$ contiene también 147 enteros.

Así que necesariamente están presentes todos ellos y además aparecen todos al menos una vez.

Preguntas

1.- ¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

Entendemos que seguimos manteniendo la estructura general de la expresión (cifra operador cifra operador cifra operador cifra operador cifra), pero permitimos repetir cifras, y permitimos repetir operadores.

En ese caso, siguen habiendo 5 posiciones de cifra y cada una puede tomar cualquiera de las 9 cifras disponibles, porque se permite la repetición, por lo tanto en este caso tenemos que el número total de posibilidades viene dado por variaciones con repetición:

$$VR(9,5) = 9^5 = 59\,049$$

Para los operadores, tenemos 4 posiciones y cada una puede tomar cualquiera de los 4 operadores, así que también vendrían dadas por variaciones con repetición de cuatro elementos tomados de cuatro en cuatro:

$$VR(4,4) = 4^4 = 256$$

Así el número total de expresiones posibles quedaría como:

$$9^5 \times 4^4 = 59\,049 \times 256 = 15\,116\,544$$

Número de expresiones posibles, sin tener en cuenta las restricciones de no repetición.

2.- ¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?

Teniendo en cuenta las restricciones listadas arriba.

Por un lado debemos elegir 5 cifras distintas de entre las 9 disponibles y además colocarlas en orden. El número de formas posibles es una variación sin repetición:

$V(n, k) = \frac{n!}{(n-k)!}$, siendo $n = 9, k = 5$, por lo que:

$$V(9,5) = \frac{9!}{(9-5)!} = 15120$$

Por otro lado, los cuatro operadores deben aparecer exactamente una vez. La forma de combinarlos es una variación sin repetición, tomando en este caso $n = 4, k = 4$, por lo que:

$$V(4,4) = \frac{4!}{(4-4)!} = \frac{4!}{0!} = \frac{4!}{1} = 4! = 24$$

Por último, lo combinamos y vemos que por cada ordenación posible de las cifras existen 24 formas de colocar los operadores. Por tanto:

$$15120 \times 24 = 362880$$

Por tanto, teniendo en cuenta todas las restricciones del problema existen 362880 expresiones válidas posibles.

Comparamos el caso sin restricciones con el caso con restricciones, temenos que las restricciones reducen el espacio de posibilidades en:

$$\frac{\text{sin restricción}}{\text{con restricción}} = \frac{15116544}{362880} \approx 41.66$$

El problema con restricciones tiene aproximadamente 42 veces menos posibilidades que el problema sin restricciones.

3.- *¿Cual es la estructura de datos que mejor se adapta al problema? Argumenta la respuesta(Es posible que hayas elegido una al principio y veas la necesidad de cambiar,argumenta)?*

Inicialmente escogimos tuplas para representar las expresiones, ya que mantienen el orden de cifras y operadores. Y para almacenar los resultados obtenidos usamos un conjunto (set), porque evita duplicados y permite búsquedas eficientes. Aunque podíamos haber utilizado una lista para guardar los resultados, pero nos decantamos por el conjunto por el elevado número de repeticiones. Por tanto, creemos que la combinación de tuplas y conjuntos se adapta muy bien a la resolución del problema mediante fuerza bruta.

4.- *¿Cual es la función objetivo?*

En principio, creemos que el problema original no tiene una función objetivo de optimización. El problema consiste en encontrar una expresión válida que produzca una cantidad dada que podría modelarse mediante la función:

$$f(B) = A$$

donde B es una expresión que cumple las restricciones del problema y A el valor buscado. Por tanto, se trata inicialmente de un problema de búsqueda bajo restricciones y no de optimización.

5.- *¿Es un problema de maximización o minimización?*

Cómo ya dijimos en la respuesta a la anterior pregunta, en principio el problema no es un problema de maximización ni de minimización, sino un problema de búsqueda bajo restricciones. El objetivo consiste en encontrar una expresión válida que produzca una cantidad dada cumpliendo simultáneamente todas las restricciones. No existe inicialmente una función objetivo que deba optimizarse. La búsqueda del máximo y

mínimo simplemente se usan para encontrar los valores extremos que pueden alcanzarse dentro del conjunto de soluciones válidas.

6.- Diseña un algoritmo para resolver el problema por fuerza bruta

Python script: pregunta3_fuerza_bruta.py

Hemos diseñado un algoritmo para resolver el problema por fuerza bruta, para ello hemos generado todas las expresiones válidas posibles. Teniendo en cuenta que cada expresión ha de cumplir las restricciones impuestas, esto es, está formada por cinco cifras distintas tomadas del conjunto $\{1, \dots, 9\}$ y por cuatro operadores $+, -, *, /$, usados solamente una vez. Para ello se emplean permutaciones de cifras y permutaciones de operadores. Cada expresión se evalúa respetando la prioridad normal de las operaciones. Si el resultado obtenido es entero, se almacena en un conjunto para evitar duplicados. Al finalizar, se obtiene el valor mínimo, el valor máximo y se comprueba si están presentes todos los enteros comprendidos entre ambos. El algoritmo evalúa $P(9,5) \cdot 4! = 362\,880$ expresiones, por lo que, tiene un tamaño relativamente pequeño y permite que este tipo de algoritmo sea viable.

7.- Calcula la complejidad del algoritmo por fuerza bruta

El algoritmo de fuerza bruta genera todas las expresiones válidas posibles. Como ya vimos en la respuesta a la pregunta 2, existen $VR(9,5) = 15\,120$ formas de seleccionar y ordenar cinco cifras distintas y $VR(4,4) = 24$ formas de ordenar los operadores. Por tanto, el número total de expresiones evaluadas es: $15120 \cdot 24 = 362\,880$. Como cada expresión tiene tamaño fijo y su evaluación requiere un número constante de operaciones, la complejidad temporal del algoritmo es:

$$O(VR(9,5) \cdot 4!) = O(362880).$$

8.- *Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta.*

9.- *Calcula la complejidad del algoritmo*

10.- Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorio.

El intervalo de valores válidos está entre el mínimo (−69) y el máximo (77).

Cada dato del juego de datos de entrada es un valor objetivo que el algoritmo debe intentar obtener mediante una expresión válida. Con el siguiente código hemos generado un conjunto de datos aleatorio.

```
import random

# Valores mínimo y máximo obtenidos previamente
minimo, maximo = -69,77

# Fijamos una semilla para que el experimento sea reproducible.
random.seed(1)

# Generamos 20 valores enteros aleatorios dentro del intervalo válido.
datos_entrada = [random.randint(minimo, maximo) for _ in range(20)]

print(datos_entrada) [-35, 76, -53, -4, -39, 57, 46, 51, 28, -16, -45, 55, -62, 30, 41, -69, 45, -1, -11, -43]
```

Se ha fijado una semilla aleatoria para que el experimento sea reproducible.

El conjunto generado es:

[-35, 76, -53, -4, -39, 57, 46, 51, 28, -16, -45, 55, -62, 30, 41, -69, 45, -1, -11, -43]

Este conjunto permite probar el algoritmo con valores positivos, negativos, cercanos a cero, al máximo y el mínimo.

11.- Aplica el algoritmo al juego de datos aleatorio generado.

Python script: pregunta3_fuerza_bruta_datos_aleatorios_entrada.py

12.- Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

- Documentación oficial de Python: `itertools.permutations`.

Se ha utilizado para generar las permutaciones de cifras y operadores sin repetición.

Esta función permite obtener todas las ordenaciones posibles de una secuencia.

- Documentación oficial de Python: `fractions.Fraction`.

Se ha utilizado para realizar cálculos exactos con fracciones y evitar errores de

precisión asociados al uso de números decimales de tipo float, especialmente en expresiones con divisiones.

- Documentación oficial de Python: random.

Se ha usado para generar el juego de datos aleatorio de entrada, formado por valores objetivo dentro del intervalo de resultados posibles. En concreto, se emplea una semilla para que el conjunto generado sea reproducible.

13.- Describe brevemente en unas líneas como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño.

Una forma de avanzar en el estudio del problema sería analizar qué ocurre al aumentar el número de cifras y operaciones que se permiten en cada expresión. Previsiblemente, esto haría que el número de combinaciones creciera rápidamente y en algún momento el algoritmo por fuerza bruta dejaría de ser tan eficiente. Se podría estudiar en qué momento en función del número de cifras u operaciones el algoritmo por fuerza bruta deja de ser una buena opción.

También sería interesante estudiar una variante en la que se permitan paréntesis. En el problema actual se respeta la prioridad habitual de las operaciones, pero no se cambia el orden mediante paréntesis, igual que en la opción anterior el número de posibilidades aumentaría rápidamente, ya que habría que considerar diferentes formas de agrupar las operaciones. Esto haría el problema mucho más complejo

Otra posible ampliación sería permitir que las cifras y/o los operadores puedan repetirse, o incluso añadir nuevos operadores como potencias, raíces o módulo. Esto cambiaría completamente el conjunto de valores alcanzables y obligaría a rediseñar parte del algoritmo.

Desde el punto de vista de la pura implementación algorítmica, se trataría de sustituir la técnica de la fuerza bruta por técnicas más avanzadas, como por ejemplo backtracking con poda. Estas técnicas más avanzadas permitirían evitar cálculos repetidos y reducir el número de ramas exploradas, especialmente si el problema aumenta de tamaño.